Detecting Harmful LLM-Agent Commands in Industrial IoT with Task-Aware Graph Neural Networks

Billal Mokhtari, Yulliwas Ameur, and Samia Bouzefrane

***Abstract*—Large language model (LLM) agents are starting to operate industrial Internet of Things (IIoT) plants. They act through edge gateways that turn tool calls into fieldbus writes to programmable logic controllers. A misled or hijacked agent can issue commands that are valid, authorized and inside its allow-list. The hardest ones are *context-shifted*: the same tool call with the same value occurs in normal operation, but under another task or another process phase. Network, agent and process monitors each miss part of this context. We link tasks, agents, tool calls, network hosts, controllers, process tags and production stages in one dynamic heterogeneous graph built at the gateway. A temporal graph neural network learns, from benign operation only, how likely each command is given its task and the current plant state. The anomaly score is the surprise of the command, computed before its effect appears. An independent red-team agent, given only MITRE ATT&CK for ICS techniques, produces the attacks. A pre-registered oracle labels them. It checks process outputs against operating limits that the detector never sees. On a battery-manufacturing testbed, we compare the model with a phase-aware value table, a tabular model given the same features, an LLM judge, and public network and agent detectors. We also measure the evasion rate of an effect-preserving adaptive attacker and the cost of running the monitor on the gateway. [One sentence (at most 25 words) with the measured result on context-shifted commands, positive or negative.]**

***Index Terms*—Industrial Internet of Things, IoT security, cyber-physical systems, IoT data analytics, edge computing, graph neural networks, large language models.**

I. Introduction

INDUSTRIAL Internet of Things (IIoT) plants are being connected to large language model (LLM) agents. An LLM agent is a program that uses a language model to plan steps and call tools. In a factory, an edge gateway exposes the tools. The gateway reads process values and writes setpoints to programmable logic controllers (PLCs) over Modbus TCP or OPC UA. Recent prototypes let agents control manufacturing cells in this way [1]–[4].

An agent acts with the credentials of a trusted operator. Prompt injection, a compromised tool or a poisoned memory can redirect it [5]. It can also simply err. Tool-using agents can turn PLC access into sustained physical impact [6]. The resulting commands are valid fieldbus writes, sent by an authorized gateway, to an allowed register.

Current defenses each see one side. Agent guards check tool calls against task-derived permissions, learned execution boundaries or predicted risk [7]–[10]. They do not know the state of the physical process. Network detectors model hosts, flows and protocol semantics [11]–[13]. They do not know which task the agent was given. Process monitors model sensor time series or state-aware invariants [14], [15]. They do not know who issued a command or why.

These blind spots meet in one class of commands that we call *context-shifted*. A context-shifted command has a tool, a target tag and a value that all occur in normal operation. Its context makes it harmful. An example is a coating-speed write that is routine during steady coating but harmful during dryer warm-up. No single view separates such a command from its benign twin (Observation 1). The evidence is spread over the task, the agent, the gateway, the controller, the tag and the production stage. These relations change every second. Graph representation learning suits such relational and time-varying IoT data [16], [17].

We ask one question. *Does a graph that links the agent's task to the plant state detect context-shifted commands better than single-view detectors?* The model is trained on benign data only. An oracle whose limits it never sees judges it.

Contributions. The paper makes three contributions.

1) **Problem and test slice.** We define context-shifted commands (Definition 1). We turn the definition into a measurable test slice: red-team commands whose tool, tag and value bin also occur in benign training data.
2) **Task-conditioned command likelihood at the edge.** We model an agent-operated IIoT cell as a dynamic heterogeneous graph built from the gateway's logs and the fieldbus. A heterogeneous temporal graph neural network (GNN) learns, from benign operation only, to predict each command from its task and the plant subgraph. A command is scored by its surprise, before its effect appears. The model needs no attack labels and runs on the gateway's CPU.
3) **A test that can fail.** Attacks come from an independent red-team LLM agent. A pre-registered oracle on process outputs provides the labels. The baselines include a phase-aware value table, a tabular model given the same features, an LLM judge, and public network and agent detectors. An adaptive attacker that must keep the

B. Mokhtari and S. Bouzefrane are with the CEDRIC Laboratory, Conservatoire National des Arts et Métiers (Cnam), 75003 Paris, France.

Y. Ameur is with Efrei Research Lab, Université Paris-Panthéon-Assas, 94800 Villejuif, France, and with the CEDRIC Laboratory, Cnam, 75003 Paris, France.

Corresponding author: B. Mokhtari ([e-mail]; ORCID [0000-000X-XXXX-XXXX]).

This work was supported by the European Union's Horizon Europe programme through the BATTwin project under Grant Agreement 101137954.

physical effect tests robustness. Data, oracle and code are released.

The architecture is standard: heterogeneous graph transformers [18] and gated recurrent memory, as in TH-GNN [19]. Our contribution is the problem, the training signal and the test.

Scope. We do not study *how* an agent is hijacked; the detector is agnostic to the cause. We exclude attacks that falsify sensors to deceive the agent; those need physics-based defenses.

II. Related Work

A. Guarding LLM Agents

Most agent guards work at the tool-call boundary. AIR-Guard derives step-level permissions from the task and simulates sensitive side effects [7]. Task Shield checks that each call serves the user's task [20]. AttriGuard re-runs the agent with weakened observations to test whether a call stems from the user's request [10]. Agent-Sentry learns the boundaries of benign executions from action sequences and argument provenance [8]. AgentSpec enforces user-written runtime rules [21]. ProbGuard abstracts traces into symbolic states, fits a Markov chain and intervenes when the predicted probability of staying safe drops [9]. DreamGuard predicts risk with a learned world model of the trajectory [22]. SafeAgent, DriftNet and TraceAegis model the session state or the trajectory [23]–[25]. AgentShield-G and AegisMCP score tool calls on typed session graphs [26], [27]. Graph monitors for multi-agent systems learn on utterance graphs [28]–[30]. RoboGuard grounds an LLM robot's plan in a world model and checks it against safety rules [31]. Shahid and Mallavrapu show indirect prompt injection against LLM copilots in industrial control systems [5]. None of these guards encodes controller registers, process phases or industrial traffic in a learned model.

B. Context-Aware Control in ICS

Industrial security has long checked commands against plant state. CPS enforces stateful policies on control-device usage [32]. CPAC checks control commands against a physical model before they reach the plant [33]. SAIN mines invariants tied to PLC program states and tightens detection bounds [15]. These systems need hand-written policies or mined invariants on the process. They do not know the operator's task. Our phase-aware value table (B0) is a learned analogue of state-aware invariants; it tells whether the task adds anything.

C. Graph-Based Intrusion Detection for IoT and IIoT

E-GraphSAGE classifies flow edges with a GNN [11]. Later work adds self-supervision [34], hierarchical traffic graphs [12], dynamic attention [35] and causal counterfactuals [36]. Malladi and Thamilarasu combine protocol-aware semantics with a mixture of graph experts for IoT intrusion detection [13]. It is the closest IoT graph detector we found, but its graph has no task or process-stage node. Our group studied GNN attack detection for IoT in [37]. GDN and IstGPT learn sensor graphs for process anomalies [14], [38]; they flag a deviation after it appears, while we score a command before its effect. Testbeds that map attacks to MITRE ATT&CK for ICS exist [39], but they have no agent.

[Architecture figure, one column. Left: LLM operator agent and red-team agent (Cnam GPU server). Middle: edge IIoT gateway with the six tools, the allow-list, the log, and our monitor (graph builder + GNN scorer) on the same CPU. Right: cell network with HMI, EWS, three PLCs and their tags, grouped by stage. Below: process twin used only by the oracle, outside the monitor's inputs. Vector PDF.]

Fig. 1. IIoT cell, edge gateway and monitor. The oracle's process twin is outside the monitor's inputs.

D. Robustness of Graph Detectors

GNN detectors are evaded by structure perturbations [40], flow injection [41] and node injection [42]. Realistic attacks must respect domain constraints [43], [44]. We add a cyber-physical constraint: the attack must keep its physical effect.

E. Positioning

Table I compares the closest works. No prior work we found conditions a learned command score on both the task and the plant state. None trains without attack labels while an external oracle labels the test.

III. System and Threat Model

A. IIoT System Model

We consider one production cell of a battery-electrode line (Fig. 1). It has three stages: slurry mixing, coating with drying, and slitting. Each stage has an OpenPLC controller [45] that exposes coils and holding registers over Modbus TCP. We call each register or coil a *tag*. An HMI and an engineering workstation share the cell network.

An LLM operator agent assists the human operator. It receives *tasks*, such as "adjust the coating speed for work order 42". It acts only through an *edge IIoT gateway*. Northbound, the gateway exposes six tools: read a tag, write a tag, read a work order, read a stage status, load a recipe and acknowledge an alarm. Southbound, it translates each call into Modbus transactions. It logs every call with its task identifier and enforces a static allow-list of tags per task type. An OPC UA gateway would only change the southbound adapter.

B. Defender Model

The monitor runs on the edge gateway. It observes the gateway log, the Modbus transactions, flow records of the cell network and PLC tag values sampled at 1 Hz. It knows the

TABLE I
POSITIONING AGAINST THE CLOSEST WORKS. ✓: PRESENT; ○: PARTIAL; –: ABSENT; ?: NOT STATED IN THE SOURCES WE CHECKED. "TASK": THE SCORE DEPENDS ON THE ASSIGNED TASK. "PLANT": THE SCORE DEPENDS ON PROCESS STATE. "NO ATTACK LABELS": TRAINED ON BENIGN DATA ONLY. "EXT. ORACLE": TEST LABELS FROM A SOURCE OUTSIDE THE MODEL'S INPUTS.

Work	Task	Plant	Network	Learned graph	No attack labels	Ext. oracle	Adaptive attacker
AIRGuard [7] (2026)	✓	–	–	–	✓	–	○
Agent-Sentry [8] (2026)	○	–	–	–	✓	–	–
ProbGuard [9] (2025)	○	○	–	–	○	–	–
RoboGuard [31] (2026)	✓	○	–	–	✓	–	✓
SAIN [15] (2024)	–	✓	–	–	✓	–	?
Malladi & Thamilarasu [13] (2026)	–	–	✓	✓	?	–	?
AegisMCP [27] (2025)	–	○	○	○	✓	–	–
This work	✓	✓	✓	✓	✓	✓	✓

static topology: which PLC controls which tag, and which tag belongs to which stage. It does *not* receive recipes, operating limits or process outputs computed by the twin. It raises alarms; it does not block traffic in this study.

C. Attacker Model

T1: Misled or hijacked agent. For part of a session, the agent pursues a harmful goal. We do not model the cause. The agent keeps its identity and the gateway, and can only call the six tools. Its writes may stay inside the allow-list.

T2: Adaptive attacker. The attacker acts through T1, knows the architecture and features, and can query the detector score (grey-box). It may only apply *realizable edits*: insert benign-looking reads, interleave benign commands from past sessions, split one change into smaller ones, and shift commands in time. Every edit must keep the physical effect defined by the oracle (Section V-B). The attacker has a budget of B extra events per episode.

Out of scope. Network attackers without the agent, poisoning of the detector's training data, compromise of the gateway or the monitor, sensor falsification and firmware attacks.

D. Context-Shifted Commands

Let a command be $c = (a, \kappa, v, t)$: tool a, target tag κ, value v and time t. Let $\mathrm{bin}(v)$ map a value to one of Q quantile bins learned on benign data. Let $\mathcal{C}_{\text{benign}}$ be the set of triples $(a, \kappa, \mathrm{bin}(v))$ observed in the benign training data.

Definition 1 (Context-shifted command)**.** *A command c is context-shifted if $(a, \kappa, \mathrm{bin}(v)) \in \mathcal{C}_{benign}$, the allow-list permits it for its task, and the oracle labels it harmful.*

Observation 1 (Single-view blindness)**.** *A detector whose score depends on a command only through $(a, \kappa, \mathrm{bin}(v))$ and agent-side history gives a context-shifted command the same score as a benign command with the same history and triple. The same holds for a network detector that sees the Modbus write, and for a process detector before the effect appears.*

The observation is elementary; we do not claim it as a result. It explains why we report the slice separately. The open question is empirical: can a learned graph model recover the missing context, beyond what a phase-aware value table already gives?

TABLE II
GRAPH SCHEMA. NO FEATURE CONTAINS A RECIPE VALUE, AN OPERATING LIMIT OR A TWIN OUTPUT.

Node type	Features
Task	task type, target stage, requested tag set; no value
Agent	session age, tool-call rate
Gateway	call rate, error rate
Host	role (HMI, EWS, gateway, unknown), flow statistics
PLC	request rate
Tag	type, current value, rolling mean and standard deviation
Stage	phase (idle, ramp, steady, transition), time in phase
Edge type	**Features**
assigned (Task→Agent)	time since assignment
invokes (Agent→Gateway)	tool identifier
writes / reads (Gateway or Host→Tag)	Modbus function code, value, change from previous value
flow (Host→Host)	NetFlow-style features [46]
controls (PLC→Tag), belongs (Tag→Stage)	static

IV. METHOD

A. Task–Plant Graph

The gateway builds snapshots $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ over sliding windows of length $\Delta = 10$ s with stride 5 s. Table II lists node and edge types.

Causal scoring rule. The score of a command at time t uses only events with timestamps strictly before t, plus the command's own fields. The effect of a command is therefore never visible when it is scored.

B. Task-Conditioned Command Likelihood

Encoder. Each node type has a linear projection to $d = 64$. Two heterogeneous graph transformer (HGT) layers [18] aggregate typed messages within each snapshot. A per-node GRU carries embeddings across snapshots: $\mathbf{m}_v^t = \mathrm{GRU}(\mathbf{h}_v^t, \mathbf{m}_v^{t-1})$. New nodes start from a zero memory.

Training signal. The model is trained on benign operation only. For each write command c issued under task τ, it predicts the target tag and the value bin from the context:

$$p_\theta(\kappa, \mathrm{bin}(v) \mid \mathbf{m}_\tau^{t^-}, \mathbf{m}_{\text{agent}}^{t^-}, \mathbf{m}_{\text{stage}(\kappa)}^{t^-}, \mathbf{m}_\kappa^{t^-}), \qquad (1)$$

where t^- is the last snapshot before t. The head is a two-layer perceptron with a softmax over allowed (tag, bin) pairs. The loss is the negative log-likelihood.

Score. The anomaly score is the surprise $s(c) = -\log p_\theta(\kappa, \mathrm{bin}(v) \mid \cdot)$. A context-shifted command has a frequent triple but an unlikely context. Its surprise is high only if the model uses the task and the stage. The alarm threshold is set on benign validation data to a 1% false positive rate.

Hyperparameters. Defaults: $Q = 8$ bins, $d = 64$, 2 HGT layers with 4 heads, Adam with learning rate 10^{-3}, batch of 32 snapshots, early stopping on validation likelihood. Training runs on one RTX 6000 Ada GPU. [Report the final values, the search grid if any (at most 2 values per parameter, chosen on benign validation data only), training time and number of parameters.]

Edge deployment. The gateway updates the current snapshot incrementally and scores each write before forwarding it. [On the gateway VM restricted to 4 CPU cores and 4 GB of memory: median and 99th-percentile latency per command ([X] ms, [X] ms), peak memory ([X] MB), model size ([X] kB).]

C. Adaptive, Effect-Preserving Attack

The attacker solves

$$\min_{\delta \in \mathcal{R}_B(e)} \max_{c \in e \oplus \delta} s_\theta(c) \quad \text{s.t.} \quad \mathrm{Oracle}(e \oplus \delta) = \text{harmful}, \qquad (2)$$

where e is an episode and $\mathcal{R}_B(e)$ is the set of realizable edit sequences with at most B added events. The oracle is evaluated by replay. The attacker greedily applies the edit that lowers the maximum score most. We use $B \in \{0, 4, 8\}$ and report the share of harmful episodes that evade each detector at its 1% false-positive threshold. [Y. Ameur implements the edit generator and the greedy search on Billal's replay interface and runs it on 50 harmful episodes per detector.]

V. Evaluation Protocol

A. Capture

The capture uses the virtual BATTwin testbed of our earlier dataset (OpenPLC, Modbus TCP, GNS3, pfSense) [47]. The gateway, the agents and the capture scripts are new. An open-weight LLM operator agent [model name and size, e.g., 7–8B] operates the cell through the gateway; it runs on the Cnam GPU server. A task generator issues routine tasks from work orders, including unusual but legitimate operator overrides. The capture runs unattended. Target: at least 8 h of benign operation [achieved: X h, Y commands].

The data are emulated, not synthetic. Modbus traffic, PLC logic, gateway logs and agent tool calls come from the real software stack. Only the plant physics is simulated. We discuss this limit in Section VII.

Independent attack generation. At random times, a red-team agent from a different LLM family [model name] replaces the operator agent. It receives only (i) the text of one of four MITRE ATT&CK for ICS techniques (T0836 Modify Parameter, T0855 Unauthorized Command Message, T0831 Manipulation of Control, T0821 Modify Controller Tasking) and (ii) the plant documentation given to the operator agent. Its goal is to degrade the process without being stopped by the allow-list. The authors write no attack script.

B. Oracle on Process Outputs

Decoupling from the benign generator. The task generator draws setpoints from work orders and recipes. The oracle does not use recipes. It checks *process outputs*, such as dryer air temperature, web tension and coated-layer thickness, against the twin's operating limits. These limits come from the equipment specification, not from the recipe book. A benign command can therefore violate them, for example an override at the wrong moment, and a red-team command can stay inside them. We report the share of operator commands that the oracle flags. We also report the phase-aware value table B0, which learns from benign data what the recipes allow in each phase. If B0 matches our model on the slice, the task adds nothing beyond recipe knowledge.

Primary oracle (twin replay). After each episode, the process twin replays the command sequence. An episode is harmful if a process output leaves its operating limits by at least τ (relative to the limit width) for at least $T_{\min}$ seconds. A command is harmful if removing it and replaying brings all outputs back inside their limits.

Fallback oracle (defined now). If the twin cannot replay command sequences on 17/10/2026, we use the logged tag values of the capture itself. An episode is harmful if a logged process tag leaves its operating limits for at least $T_{\min}$ seconds within 120 s of a red-team command. Command-level labels then use the last write to a tag of the same stage before the violation. This oracle is weaker: it cannot test counterfactual removals. The adaptive attacker then checks the effect by re-running the edited episode on the testbed, which costs more time; B is then limited to $\{0, 4\}$.

[By 17/10/2026: primary or fallback oracle; list of process outputs, their limits, τ and $T_{\min}$ as a supplementary table.] [Y. Ameur writes and files the OSF registration (oracle, limits, hypotheses, analysis script): DOI and date, no later than 22/10/2026.]

The detector never receives operating limits, recipes or twin outputs (Table II), and it scores each command before its effect. Red-team episodes that cause no violation are labeled benign; operator episodes that cause one are labeled harmful.

C. Pilot and Go/No-Go

A pilot runs before the main capture: 1 h of benign operation and about 30 red-team episodes [pilot run by 25/10/2026]. The go criterion has two parts. First, at least 50 context-shifted commands. Second, a depth-3 decision tree on the triple $(a, \kappa, \mathrm{bin}(v))$ plus the stage phase and the task type stays below 0.9 AUPRC on them, with a chronological split of the pilot. A tree on the triple alone cannot separate the slice by construction; adding phase and task type tests whether the slice is trivially separable. [Y. Ameur computes the tree and B0 on the pilot: [X] context-shifted commands; tree AUPRC [X]; B0 AUPRC [X]; decision go/no-go.] If the tree exceeds 0.9, the red-team prompt and the task generator are revised

once. If it still exceeds 0.9, we report the slice as too easy and the paper becomes a negative result.

Power analysis. The main test (H1) is a paired McNemar test on episode-level detection. Assume discordant proportions of 0.15 and 0.05. Then 155 harmful episodes give 80% power at $\alpha = 0.05$. The campaign runs until 160 harmful episodes are collected, with a cap of 300 episodes [achieved: X harmful out of Y episodes; Z context-shifted commands].

D. Splits

We never use random splits [48]. Training uses the first 70% of benign time, validation the next 10%, and test the last 20% plus all red-team episodes. Thresholds and hyperparameters never see red-team data.

E. Baselines

All baselines receive the same streams and the same causal scoring rule. External detectors use their public implementations without architectural change.

- **B0 Phase-aware value table**: for each (tag, stage phase), the set of value bins seen in benign training; score 1 if the bin is unseen, else the bin's negative log-frequency. It is a learned analogue of state-aware invariants [15]. [Implemented by Y. Ameur.]
- **B1 Allow-list**: the gateway's own policy.
- **B2 Same-feature tabular model**: XGBoost predicts the same (tag, bin) target from the same features, flattened per command. Its score is the same surprise. If our model does not beat B2, the graph adds nothing.
- **B3 Network view**: E-GraphSAGE [11] on flows and Modbus transactions.
- **B4 Agent view**: AIRGuard [7], configured with our six tool specifications and run with the operator's LLM. [Y. Ameur writes the Modbus tool specifications and the log adapter; no change to its code.]
- **B5 LLM judge**: the operator's LLM receives the task text, the phase of each stage and the proposed command, and returns a risk score from 0 to 10. [Y. Ameur freezes the prompt before the test data are opened and runs B5.]

Ablations. Our model restricted to the agent subgraph, to the plant subgraph and to the network subgraph; without the task node.

F. Metrics and Statistics

Detection: AUROC, AUPRC, episode-level recall at 1% false positive rate and alarms per hour, on three slices: all harmful commands, those that pass the allow-list, and context-shifted commands. Robustness: evasion rate at each budget. Edge cost: latency and memory on the gateway. Statistics: 3 seeds; 95% confidence intervals by bootstrap over episodes; McNemar tests on episode-level decisions; Holm correction across baselines. [Y. Ameur writes the analysis script before the test data are opened, deposits it with the registration and runs it on Billal's score files.]

TABLE III
EPISODE-LEVEL RECALL AT 1% FPR (%) ON THREE SLICES OF THE TEST SET; MEAN AND 95% CI OVER 3 SEEDS. "ALLOW": HARMFUL COMMANDS THAT PASS THE ALLOW-LIST. "CTX": CONTEXT-SHIFTED COMMANDS.

Method	All	Allow	Ctx
B0 Phase-value table	[X]	[X]	[X]
B1 Allow-list	[X]	0	0
B2 Tabular	[X]	[X]	[X]
B3 E-GraphSAGE	[X]	[X]	[X]
B4 AIRGuard	[X]	[X]	[X]
B5 LLM judge	[X]	[X]	[X]
Ours	[X]	[X]	[X]

G. Hypotheses (pre-registered, may fail)

H1 On context-shifted commands, our model's episode-level recall at 1% false positive rate exceeds B0, B2, B3, B4 and B5 by at least 10 points (McNemar, Holm-corrected $p < 0.05$).

H2 On the same slice, the single-view ablations have AUROC at most 0.6, and removing the task node costs more AUROC than removing the network subgraph.

H3 At $B = 4$, fewer harmful episodes evade our model than B2 and B4.

If H1 fails against B0, the task adds nothing beyond phase-aware value statistics; if it fails against B2, the graph adds nothing beyond flattened context. Both outcomes are reported.

VI. RESULTS

A. Pilot and Data Summary

[Table, one column: pilot go/no-go figures; benign hours [X], commands [X], red-team episodes [X], harmful episodes [X], context-shifted commands [X], counts per technique [X]; share of operator commands flagged by the oracle [X %].]

B. Detection (H1)

Table III reports the main comparison.

[Two to four sentences: H1 accepted or rejected with the Holm-corrected McNemar p-values; alarms per hour of each method.]

C. Where Does the Signal Come From? (H2)

[Small table: AUROC on the context-shifted slice for the agent-only, plant-only and network-only models and without the task node: [X] each. Two sentences on H2.]

D. Adaptive Attacker (H3)

[Evasion rate at each budget for B2, B4 and ours: [X %] each, with 95% CIs, in a 3-row table.]

E. Cost at the Edge

[One sentence with the latency, memory and model size measured in Section IV-B.]

VII. Threats to Validity

Emulated plant. Traffic, gateway logs and agent behavior come from the real software stack, but plant physics comes from the testbed's process twin. A real line may show other dynamics. The released data let others test transfer.

Oracle. The oracle uses operating limits on process outputs, kept apart from the recipes that drive the benign generator. Some harmful effects, such as equipment wear, stay outside these limits. Under the fallback oracle, command-level labels rely on a timing rule and are less precise.

Emulated misuse. Hijacks are emulated by switching to a red-team agent. Real hijacks may leave other traces. A different model family and technique-only prompts reduce, but do not remove, the risk that the attacks favor our detector.

Benign coverage. A model trained on benign data flags rare legitimate overrides. We include such overrides in the benign capture and report alarms per hour.

Scale. One agent, one cell and one fieldbus. Multi-agent, multi-cell and OPC UA settings are left for future work.

VIII. Conclusion

LLM agents can issue harmful industrial commands that look normal to every single-view defense. We defined these context-shifted commands, built a graph at the IIoT gateway that links the agent's task to the plant state, and tested a task-conditioned command likelihood against an oracle on process outputs. [One or two sentences with the measured outcome of H1 and H2, positive or negative.] Data, oracle and code are released.

Acknowledgment

The authors used Claude (Anthropic) to draft and edit the text of Sections I–IV and to check references, in line with the IEEE policy on AI-generated text [49]. The authors reviewed and revised all text. No result, figure or code reported in the paper was generated by an AI system. The LLM agents in the capture are part of the experimental object. [Confirm before submission.]

Author Contributions

B. Mokhtari: conceptualization, data curation, investigation, software (gateway, capture, model, B1–B3), validation, visualization, writing – original draft (results). Y. Ameur: conceptualization, methodology, software (B0, B4, B5, adaptive attacker), formal analysis (pre-registration, statistics), writing – original draft (Sections I–IV, prepared with AI assistance as stated above), writing – review and editing. S. Bouzefrane: funding acquisition, project administration, resources, supervision, writing – review and editing.

Data Availability

The capture, the oracle, its OSF registration, the gateway, the graph construction code and the model code are archived on Zenodo under an open license [Zenodo DOI]. Testbed credentials and network secrets are removed before release.

Conflict of Interest

The authors declare no competing interests.

References

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, 2025.

[2] J. Vyas and M. Mercangöz, "Autonomous control leveraging LLMs: An agentic framework for next-generation industrial automation," arXiv:2507.07115, 2025.

[3] O. Hornyák and M. Kernács, "PLC-integrated large language models for interpretable control in industrial automation," in *Proc. Int. Carpathian Control Conf. (ICCC)*, 2026.

[4] I. B. R. Nogueira and S. Skogestad, "A systematic approach to multi-agent AI from advanced regulatory control theory: Safe and auditable LLM operator agents for process control," arXiv:2606.30877, 2026.

[5] S. Shahid and D. Mallavrapu, "Indirect prompt injection against LLM copilots in industrial control systems," in *Design of Cyber-Secure Critical Infrastructure*, ser. Commun. Comput. Inf. Sci. Springer, 2026, pp. 3–21.

[6] Y. Zhou, J. Zheng, Q. Jiang, L. Du, H. Liu, L. Wu, S. Zhao, M. Liu, and R. Deng, "PLCBench: Can autonomous LLM agents turn PLC access into sustained physical impact?" arXiv:2608.26882, 2026.

[7] S. Qin, H. Zhuang, Y. Zhou, Y. Han, and X. Zhang, "AIRGuard: Guarding agent actions with runtime authority control," arXiv:2605.28914. Code: https://github.com/Sophie508/AIRGuard, 2026.

[8] R. Sequeira, S. Damianakis, U. Iqbal, and K. Psounis, "Agent-Sentry: Bounding LLM agents via execution provenance," arXiv:2603.22868, 2026.

[9] H. Wang, C. M. Poskitt, J. Wei, and J. Sun, "ProbGuard: Proactive runtime monitoring for LLM agent safety via probabilistic prediction," arXiv:2508.00500, 2025.

[10] Y. He, H. Zhu, Y. Li, S. Shao, H. Yao, Z. Liu, and Z. Qin, "AttriGuard: Defeating indirect prompt injection in LLM agents via causal attribution of tool invocations," arXiv:2603.10749, 2026.

[11] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Netw. Oper. Manage. Symp. (NOMS)*, 2022.

[12] X. Kong, Z. Song, X. Ye, J. Jiao, H. Qi, and X. Liu, "GRID: Graph-based robust intrusion detection solution for industrial IoT networks," *IEEE Internet Things J.*, 2025.

[13] G. Malladi and G. Thamilarasu, "Protocol-aware semantic mixture-of-experts graph learning for intrusion detection in IoT networks," in *Proc. 35th Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2026, pp. 1–9.

[14] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, 2021.

[15] S. G. Abbas, M. O. Ozmen, A. Alsaheel, A. Khan, Z. B. Celik, and D. Xu, "SAIN: Improving ICS attack detection sensitivity via state-aware invariants," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 6597–6613.

[16] G. Dong, M. Tang, Z. Wang, J. Gao, S. Guo, L. Cai, R. Gutierrez, B. Campbell, L. E. Barnes, and M. Boukhechba, "Graph neural networks in IoT: A survey," *ACM Trans. Sensor Netw.*, 2023.

[17] S. K. Moorthy and J. Jagannath, "Survey of graph neural network for Internet of Things and NextG networks," *IEEE Commun. Surveys Tuts.*, 2025.

[18] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *Proc. Web Conf. (WWW)*, 2020.

[19] S. Swarup, D. Shrivastava, and R. Thakur, "TH-GNN: Heterogeneous temporal graph neural networks for LLM-agent shilling attack detection," arXiv:2608.20376, 2026.

[20] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025, pp. 29 680–29 697.

[21] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," arXiv:2503.18666, 2025.

[22] W. Lin, C. Yu, X. Lin, S. Cao, X. Chen, L. Xue, L. Yu, L. Sha, and C. Wu, "DreamGuard: Efficient runtime guardrail for LLM agents via risk-aware world model," arXiv:2608.05695, 2026.

[23] H. Liu, E. Ilyushin, J. Ni, and M. Zhu, "SafeAgent: A runtime protection architecture for agentic systems," arXiv:2604.17562, 2026.

[24] A. Pinjari and M. P. Saint-Germain, "DriftNet: A dual-head trajectory transformer for detecting and localizing prompt injection in LLM agents," arXiv:2609.10892, 2026.

[25] J. Liu, B. Ruan, X. Yang, Z. Lin, Y. Liu, Y. Wang, T. Wei, and Z. Liang, "TraceAegis: Securing LLM-based agents via hierarchical and behavioral anomaly detection," arXiv:2510.11203, 2025.
[26] V. R. Divi and S. Gullapalli, "Graph-based detection of prompt injection and tool misuse in MCP-enabled AI agents: The AgentShield-G guardrail model," *Int. J. Comput. Appl.*, vol. 187, no. 127, pp. 38–44, 2026.
[27] Z. Zhan, A. Al Sadi, K. Li, and H. Haddadi, "AegisMCP: Online graph intrusion detection for tool-augmented LLMs on edge devices," arXiv:2510.19462, 2025.
[28] S. Wang, G. Zhang, M. Yu, G. Wan, F. Meng, C. Guo, K. Wang, and Y. Wang, "G-Safeguard: A topology-guided security lens and treatment on LLM-based multi-agent systems," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025, pp. 7261–7276.
[29] J. Pan, Y. Liu, R. Miao, K. Ding, Y. Zheng, Q. V. H. Nguyen, A. W.-C. Liew, and S. Pan, "Explainable and fine-grained safeguarding of LLM multi-agent systems via bi-level graph anomaly detection," in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2026, pp. 30 491–30 506.
[30] J. Zhou, L. Wang, and X. Yang, "GUARDIAN: Safeguarding LLM multi-agent collaborations with temporal graph modeling," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025, arXiv:2505.19234.
[31] Z. Ravichandran, A. Robey, V. Kumar, G. J. Pappas, and H. Hassani, "Safety guardrails for LLM-enabled robots," *IEEE Robot. Autom. Lett.*, vol. 11, no. 4, pp. 4649–4656, 2026.
[32] S. McLaughlin, "CPS: Stateful policy enforcement for control system device usage," in *Proc. 29th Annu. Comput. Secur. Appl. Conf. (ACSAC)*, 2013, pp. 109–118.
[33] S. Etigowni, D. J. Tian, G. Hernandez, S. Zonouz, and K. Butler, "CPAC: Securing critical infrastructure with cyber-physical access control," in *Proc. 32nd Annu. Comput. Secur. Appl. Conf. (ACSAC)*, 2016, pp. 139–152.
[34] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," *Knowledge-Based Systems*, vol. 258, p. 110030, 2022.
[35] N. Sun, L. Chen, and G. Han, "HADGA: Hierarchical attention-based dynamic GNN algorithm for IoT botnet detection," *IEEE Internet Things J.*, 2025.
[36] C. Zhang, L. Zheng, H. Huang, and C. Su, "CCG-IDS: A causal counterfactual graph-based intrusion detection system for industrial IoT," *IEEE Trans. Ind. Informat.*, 2026.
[37] R. Rezki, Y. Badr, S. Bouzefrane, F. Mourlin, and M. Yacoub, "Leveraging graph neural networks for attack detection in IoT systems," in *Proc. Int. Conf. Availability, Reliability and Security (ARES)*, 2025.
[38] Y. Zhang, N. Xi, P. Feng, S. Liu, J. Ma, Y. Shen, Y. Sun, and X. Zhou, "IstGPT: LLM-based anomaly detection for spatial-temporal graph in industrial systems," arXiv:2606.01691, 2026.
[39] J. Rahmani, K.-O. Detken, and A. Sikora, "An integrated testbed for MITRE-mapped attack emulation in industrial control networks," *Sensors*, 2026.
[40] D. Zügner, O. Borchert, A. Akbarnejad, and S. Günnemann, "Adversarial attacks on graph neural networks: Perturbations and their patterns," *ACM Trans. Knowl. Discov. Data*, 2020.
[41] M. Mouzaoui, Y. Han, G. Blanc, G. Rilling, and M. Hurfin, "Robustness evaluation of graph neural network-based network intrusion detection systems against adversarial flow injection," in *Proc. Int. Conf. Secur. Cryptogr. (SECRYPT)*, 2026.
[42] W. Liang, R. Yan, J. Cai, and Y.-G. Wang, "Single-edge node injection threats to GNN-based security monitoring in industrial graph systems," arXiv:2602.01113, 2026.
[43] R. Sheatsley, N. Papernot, M. Weisman, G. Verma, and P. McDaniel, "Adversarial examples for network intrusion detection systems," *J. Comput. Secur.*, 2022.
[44] M. Catillo, A. Pecchia, A. Repola, and U. Villano, "Towards realistic problem-space adversarial attacks against machine learning in network intrusion detection," in *Proc. Int. Conf. Availability, Reliability and Security (ARES)*, 2024.
[45] T. Alves and T. Morris, "OpenPLC: An IEC 61131-3 compliant open source industrial controller for cyber security research," *Computers & Security*, vol. 78, pp. 364–379, 2018.
[46] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Netw. Appl.*, 2022.
[47] B. Mokhtari, S. Bouzefrane *et al.*, "An industrial control system dataset collected from a testbed for battery manufacturing under normal and attack conditions," Manuscript under review at *Scientific Data*, 2026.
[48] S. Zavrak, "Content-aware attack detection in LLM agent tool-call traffic: An empirical study of features, architectures, and evaluation protocols," arXiv:2605.11053, 2026.
[49] IEEE, "Author guidelines for artificial intelligence (AI)-generated text," https://open.ieee.org/author-guidelines-for-artificial-intelligence-ai-generated-text/, 2026, accessed Oct. 15, 2026.

Billal Mokhtari is a Ph.D. candidate at the CEDRIC Laboratory, Cnam, Paris, France. [Degrees and research interests, 3 lines.]

Yulliwas Ameur is with Efrei Research Lab, Université Paris-Panthéon-Assas, and with the CEDRIC Laboratory, Cnam, Paris, France.

Samia Bouzefrane is a Professor at Cnam, Paris, France, and a member of the CEDRIC Laboratory.